

Operator-Level Boolean Computation with Correspondence Matrices

Typed operand frames, local operator folding, fallback semantics, and empirical boundaries

Brian Theory

Post-split revised computational draft — 22 September 2026

Abstract

Correspondence matrices (CMs) encode a binary Boolean connective as a four-bit 2×2 operator in a declared operand frame. This paper studies a compiler transformation built on that representation rather than claiming a new truth-table algebra. The compiler recognizes binary operator occurrences, records their row/column operands and polarities, transports swapped or negated occurrences into a canonical frame, and fuses tokens only after exact frame agreement. It distinguishes pure structural folding, hybrid folding through locally retabulated descendants, whole-root four-assignment retabulation, and ordinary fallback. We state a common semantic invariant and prove soundness for each admitted pair path. The implementation keeps compact tokens, framed pair surrogates, general Boolean IR, lowered programs, and dense outputs as distinct artifacts so that constant-size kernel operations are not confused with end-to-end costs. Existing exploratory S1/S2 records show that local small-function folding can reduce pre-expansion work in some strata, but a matched generic local truth-function optimizer reaches the same reduced outputs. A separately frozen ANF-dispatch study is negative overall: highly profitable selected synthetic folds do not repay dispatcher overhead on the aggregate synthetic and natural corpora. Broader current-source project benchmarks likewise emphasize task matching and strong packed/CSE controls, but they measure different endpoints and are not pair-compiler results. The confirmatory pair-compiler protocol against direct packed, prepared DAG, ROBDD, and synthesis-compatible baselines remains unexecuted. The contribution is therefore a typed frame-aware compilation rule, its sound fallback boundary, and a disciplined evaluation framework for determining when local folding amortizes in practice.

Draft status. This is the revised compiler-focused manuscript after the deliberate CM/LM publication split. The companion foundations paper *Correspondence and Logical Matrices: A Boolean Operator Calculus* owns the formula-valued LM theory, valuation, logical pairing, higher-arity tensors/maps, and the broader coherence calculus [15]. The present paper imports only the numeric binary contract required by the compiler. Protocol v3 has not yet been executed; no pair-compiler end-to-end speedup is claimed.

1 Introduction and claim boundary

A binary Boolean connective contains four output values. A correspondence matrix stores those four values as a 2×2 array whose row and column axes are tied to declared operands. In isolation, that is only a reshaping of a truth table. The computational question is different: can a compiler use the small operator as a first-class token, normalize the token’s operand frame, and combine compatible local operators before evaluating or expanding their operands?

The key example is deliberately simple. Consider

$$(X \Rightarrow Y) \hat{\Downarrow} (Y \Rightarrow X). \quad (1)$$

Both inner connectives are implication, but they do not initially refer to the same ordered operand frame. Treating their four-bit arrays as if corresponding entries described the same assignment would be a type error. Once the second occurrence is transported from frame (Y, X) to frame (X, Y) by transpose, entrywise XOR is sound:

$$[\Rightarrow] \hat{\Downarrow} [\Rightarrow]^T = [\hat{\Downarrow}]. \quad (2)$$

The resulting token represents $X \hat{\Downarrow} Y$. Equation (2) is not important because XOR of two truth tables is difficult; it is important because it makes the *admission condition* explicit. A local table may be fused only when its coordinates have a common logical meaning.

This paper therefore treats operand-frame metadata as part of the compiler type. A successful pair compilation returns not just four bits, but a token tied to canonical row and column operands. Swaps and input polarities are normalized by fixed reindexings. Compatible tokens can then be fused by an outer Boolean connective before repeated operand work or polynomial expansion. If structural recognition fails, the implementation can exactly retabulate an admitted two-variable subtree, can combine such a retabulated descendant with structural parents while marking the result hybrid, or can leave the expression on the ordinary IR path.

The component operations have substantial prior art. Boolean matrices predate this work [5]; matrix and vector logics represent logical operators algebraically [13, 8, 9]; Cheng, Zhao, and Xu explicitly combine truth vectors entrywise under an arbitrary outer Boolean operation [3]; small truth functions, NPN transformations, LUTs, and DAG-aware rewriting are standard synthesis techniques [7, 17, 14]; and bit-parallel evaluators compute several assignments simultaneously [6]. Bricken’s internally dated 1997 technical note is also a close antecedent for 2×2 logical matrices and bra–matrix–ket evaluation [1]. The present paper does not claim those ingredients as new.

The narrower contribution is computational and testable:

1. a **typed frame-aware compiler rule** in which a four-bit CM token is paired with operand metadata and normalized before same-frame fusion;
2. **sound multi-path semantics** separating pure structural compilation, local retabulation, hybrid composition, and ordinary fallback;
3. an **implementation and cost boundary** separating compact tokens, framed surrogates, general IR, lowered programs, and dense materialized outputs;
4. **matched mechanism and negative evidence**: exploratory folding can reduce pre-expansion work, an equally capable generic optimizer can reproduce the same reductions, and a frozen dispatcher study shows that local wins need not amortize; and
5. a **confirmatory methodology** that requires direct packed and sharing-aware comparators, preparation costs, reuse, output equivalence, memory, fallback, and failures to be measured on the same task.

The scientific claim does not depend on CM folding being uniquely capable of a reduction. If a generic local truth-function optimizer reaches the same reduced form, that locates the mechanism in a broader family of small-function rewrites. The remaining CM-specific question is whether the representation and its frame metadata make recognition, legality, transformation, caching, or reuse sufficiently direct and inexpensive to matter in a full compiler.

2 Minimal CM compiler contract

The companion foundations paper develops the CM/LM calculus in detail [15]. Only the following binary numeric contract is required here.

Let

$$\mathbb{B} = \{0, 1\} \cong \mathbb{F}_2, \quad (3)$$

with multiplication interpreted as conjunction and addition written as XOR, $\uparrow$. For a bit $x \in \mathbb{B}$, use the true-first state

$$|x\rangle = \begin{bmatrix} x \\ -x \end{bmatrix}, \quad \langle x| = [x \quad -x]. \quad (4)$$

For a binary logical connective variable Θ , define the compact correspondence matrix

$$[\Theta] := \begin{bmatrix} \Theta_{11} & \Theta_{10} \\ \Theta_{01} & \Theta_{00} \end{bmatrix} = \begin{bmatrix} 1\Theta 1 & 1\Theta 0 \\ 0\Theta 1 & 0\Theta 0 \end{bmatrix}. \quad (5)$$

The row and column assignment order $(1, 0)$ is part of the representation contract.

For $C \in \mathbb{B}^{2 \times 2}$, define the XOR–AND contraction

$$\mathcal{E}_C(x, y) = \langle x|C|y\rangle = \uparrow_{r,s \in \{1,0\}} \chi_x(r) \wedge C_{rs} \wedge \chi_y(s), \quad (6)$$

where $\chi_x(r) = 1$ exactly when $r = x$.

Proposition 1 (Selection). *For every binary connective Θ and $x, y \in \mathbb{B}$,*

$$\langle x|[\Theta]|y\rangle = x\Theta y. \quad (7)$$

Proof. Both states are one-hot. Every term in Equation (6) vanishes except the term at $(r, s) = (x, y)$. $\square$

The compiler packs the four coefficients into a constant-size token. That token is not by itself sufficient to authorize a rewrite. An occurrence is also typed by an *operand frame*: the row expression, column expression, their signed polarities, and the truth-state order. In the implementation considered here, a successful pair surrogate ultimately records canonical row and column variable names together with the normalized token.

3 Typed normalization and same-frame fusion

Let

$$P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = [\updownarrow]. \quad (8)$$

Define $\Theta^\tau(x, y) := y\Theta x$. With the true-first convention,

$$[\Theta^\tau] = [\Theta]^T, \quad (9)$$

$$[\Theta(\neg X, Y)] = P[\Theta], \quad (10)$$

$$[\Theta(X, \neg Y)] = [\Theta]P, \quad (11)$$

where products with P are XOR-AND products and therefore act as row/column permutations. Output negation is entrywise complement.

For a binary outer connective Φ , define same-position fusion by

$$([\Theta_1]\hat{\Phi}[\Theta_2])_{rs} := (\Theta_1)_{rs}\Phi(\Theta_2)_{rs}. \quad (12)$$

This is not matrix multiplication.

Theorem 1 (Same-frame fusion). *If $[\Theta_1]$ and $[\Theta_2]$ are expressed in the same ordered operand frame, then for all $x, y \in \mathbb{B}$,*

$$(x\Theta_1 y)\Phi(x\Theta_2 y) = \left\langle x \middle| ([\Theta_1]\hat{\Phi}[\Theta_2]) \middle| y \right\rangle. \quad (13)$$

Proof. Both sides select position (x, y) ; the right-hand side stores the outer operation on exactly those two selected coefficients. No distributive law for Φ over XOR is required. $\square$

The pointwise identity itself is classical; Cheng, Zhao, and Xu give a direct truth-vector antecedent [3]. The compiler-specific rule is therefore:

Admission rule. Transport signed/swapped occurrences to one declared frame. Apply Equation (13) only after row operand, column operand, polarity interpretation, and truth-state order agree.

Figure 1 summarizes the resulting local pipeline.

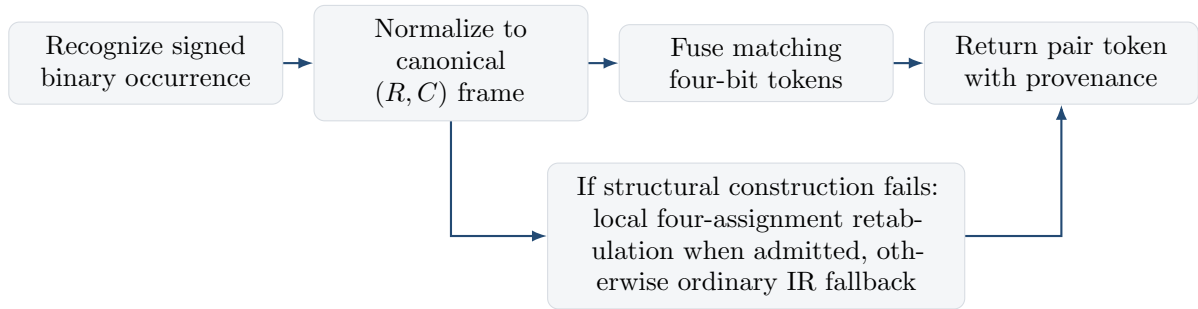


Figure 1: The compiler transformation. The operation performed on the four-bit token is trivial; the semantic work is the typed recognition and normalization that determines when fusion is legal and what fallback is required.

3.1 Why frame metadata matters

Return to Equation (1). The two implication tokens are numerically equal before typing, but the second occurrence's array coordinates refer to (Y, X) rather than (X, Y) . Naively XORing the two unaligned arrays would return the zero token. Transposition moves the second operator into the target frame, after which Equation (2) correctly yields $[\uparrow]$. The error is therefore not a deficiency of truth tables; it is a failure to track which assignment each stored bit denotes.

This distinction is closely related to established NPN-style input permutation/negation and output-negation operations [7, 17]. The present use is intentionally narrower than a general NPN canonicalizer. It treats frame transport as a local typing obligation for a particular rewrite.

4 Pair compiler and soundness

4.1 Judgment, invariant, and provenance

Let expressions use the grammar

$$e ::= Z \mid \neg e \mid e \Phi e, \quad (14)$$

where Z is a variable and Φ is one of the currently supported binary connectives AND, OR, XOR, implication, and equivalence. Fix distinct canonical variables R and C and a fixed-substitution map F that binds neither variable on a returned pair path. Write $\Gamma = (R, C, F)$.

A typed pair result has judgment

$$\Gamma \vdash e \Downarrow^p \text{Pair}(R, C, t), \quad p \in \{S, T, H\}, \quad (15)$$

with the intended invariant

$$\mathcal{E}_t(r, c) = e[r/R, c/C] \quad \text{for all } r, c \in \mathbb{B}, \quad (16)$$

interpreted after the fixed substitutions in F .

Equation (15) can be read as follows: under compiler environment Γ , expression e has been reduced to the four-bit token t in canonical frame (R, C) , and p records how that token was obtained. Equation (16) is the semantic contract. Each of the four token positions must return exactly the value of the original expression under the corresponding assignment to R and C . The pair therefore carries both a compact Boolean function and the frame metadata needed to interpret its bit positions.

The provenance values are:

- S — **structural**: the returned subtree used signed primitive recognition, token complement, and/or same-frame fusion only.
- T — **tabulated root**: the returned subtree itself was produced by evaluating all four assignments. The symbol T avoids overloading R , which already denotes the row variable.
- H — **hybrid**: structural steps occur above at least one tabulated descendant.

A top-level compilation additionally records `ordinary_fallback` when no pair judgment is derived.

4.2 Rules, built up one step at a time

The inference-rule bar is read as implication: if the premises above the bar hold, the compiler may derive the conclusion below it. The rules perform four conceptually distinct jobs: create a normalized primitive token, complement an already compiled token, fuse two already aligned tokens, and—when enabled—tabulate an exact two-variable subtree that structural recognition could not build.

Primitive normalization. Let ℓ_1, ℓ_2 be signed literals whose underlying variables are exactly R and C in either order. If the declared normalization procedure transports the primitive occurrence $\Theta(\ell_1, \ell_2)$ into the canonical frame and returns token t , then the compiler may derive

$$\frac{\text{Norm}_\Gamma(\Theta(\ell_1, \ell_2)) = t}{\Gamma \vdash \Theta(\ell_1, \ell_2) \Downarrow^S \text{Pair}(R, C, t)}.$$

Normalization consists only of the declared operand exchange and row/column polarity transforms from Equation (11). This is the structural base case: it establishes a token whose four bit positions already refer to the canonical assignments $(R, C) = (1, 1), (1, 0), (0, 1), (0, 0)$.

Negation: Equation (17). Once a child has a correct token in the canonical frame, negating the expression requires only bitwise complement of that token:

$$\frac{\Gamma \vdash a \Downarrow^p \text{Pair}(R, C, t_a)}{\Gamma \vdash \neg a \Downarrow^{\nu(p)} \text{Pair}(R, C, \neg t_a)}. \quad (17)$$

The frame does not change. Here $\neg t_a$ means complement each of the four token bits. Provenance changes only to report ancestry: $\nu(S) = S$ and $\nu(T) = \nu(H) = H$. Thus a structural NOT above a locally tabulated child is correct but explicitly reported as hybrid.

Same-frame fusion: Equation (18). Two compiled children may be fused only when their pair metadata name the same canonical frame:

$$\frac{\Gamma \vdash a \Downarrow^p \text{Pair}(R, C, t_a) \quad \Gamma \vdash b \Downarrow^q \text{Pair}(R, C, t_b)}{\Gamma \vdash a \Phi b \Downarrow^{p \sqcup q} \text{Pair}(R, C, t_a \hat{\Phi} t_b)}. \quad (18)$$

The token operation $t_a \hat{\Phi} t_b$ applies Φ independently to corresponding token positions. The repeated $\text{Pair}(R, C, \cdot)$ in the two premises is the essential admission certificate: position 11 in one child denotes the same assignment as position 11 in the other, and similarly for 10, 01, 00. Define $S \sqcup S = S$; every other combination is H because tabulated ancestry is present below a structural fusion.

Exact local tabulation. When the selected strategy permits local tabulation and the live support after fixed substitution is exactly $\{R, C\}$, define the true-first token

$$t_e := (e(1, 1), e(1, 0), e(0, 1), e(0, 0)). \quad (19)$$

Then the exact tabulation rule is

$$\frac{\text{supp}_F(e) = \{R, C\} \quad t = t_e}{\Gamma \vdash e \Downarrow^T \text{Pair}(R, C, t)}.$$

This rule re-evaluates a two-variable subtree on its complete four-assignment domain. It is therefore an exact fallback base case, not evidence that the compiler recognized the same structure syntactically.

Worked derivation. Let $\Gamma = (X, Y, \emptyset)$ and

$$e = (X \Rightarrow \neg Y) \Updownarrow (\neg Y \Rightarrow X). \quad (20)$$

In true-first order (11, 10, 01, 00), implication has token 1011₂. For the left child, the variables are already in (X, Y) order but the column literal is negated, so the column-polarity transform gives

$$X \Rightarrow \neg Y \quad \longmapsto \quad 0111_2.$$

For the right child, the source operands arrive as $(\neg Y, X)$. Operand exchange followed by polarity alignment gives

$$\neg Y \Rightarrow X \quad \longmapsto \quad 1110_2.$$

Both children now carry frame (X, Y) , so Equation (18) is legal:

$$0111_2 \Updownarrow 1110_2 = 1001_2. \quad (21)$$

Hence

$$\Gamma \vdash e \Downarrow^S \text{Pair}(X, Y, 1001_2).$$

The token 1001₂ is $[\Leftrightarrow]$ in true-first order, so the compiler has established

$$(X \Rightarrow \neg Y) \Updownarrow (\neg Y \Rightarrow X) = X \Leftrightarrow Y$$

without first expanding the source expression into an ANF or another larger symbolic representation. Applying Equation (17) once more complements 1001₂ to 0110₂, the XOR token.

The public compiler strategies are therefore:

- **pure_structural**: primitive normalization, negation, and same-frame fusion only;
- **hybrid**: the structural rules plus exact local pair tabulation when needed;
- **retabulate**: bypass recursive pair construction and tabulate the whole admitted root;
- **ordinary fallback**: leave non-admitted expressions on the general IR path.

This explicit hybrid class is important. An earlier implementation/review cycle allowed a tabulated child to be consumed by a structurally compiled parent without naming that path. The current specification records it instead of silently treating it as purely structural.

Theorem 2 (Pure structural soundness). *If $\Gamma \vdash e \Downarrow^S \text{Pair}(R, C, t)$, then Equation (16) holds and no four-assignment tabulation was used.*

Proof. Induct on the derivation. A signed primitive is correct by the frame transforms in Equation (11); complementing a correct child represents negation; and fusing two correct children with identical metadata is sound by Theorem 1. $\square$

Definition 1 (Declared pure-structural fragment). Fix canonical variables R, C . Let $\mathcal{S}_{R,C}$ be the smallest expression class containing every supported primitive $\Theta(\ell_1, \ell_2)$ whose signed literals have underlying variables exactly R, C in either order, and closed under negation and the five supported binary connectives.

Theorem 3 (Completeness for the declared pure-structural fragment). *For every $e \in \mathcal{S}_{R,C}$, the **pure_structural** strategy derives a unique canonical-frame token t such that*

$$\Gamma \vdash e \Downarrow^S \text{Pair}(R, C, t)$$

and Equation (16) holds.

Proof. Induct on the construction of e . Every primitive in $\mathcal{S}_{R,C}$ is admitted by primitive normalization. Negation preserves the frame and remains structural by Equation (17). If two children belong to $\mathcal{S}_{R,C}$, induction gives tokens in the same canonical frame, so Equation (18) applies. Uniqueness follows because a Boolean function on fixed variables R, C has one four-bit truth token in the declared true-first order. $\square$

Remark 1. This is syntactic completeness for the deliberately declared compiler fragment, not semantic completeness for all formulas with two-variable support. A semantically equivalent expression outside $\mathcal{S}_{R,C}$ may still require exact tabulation or ordinary fallback.

Lemma 1 (Exact pair tabulation). *Suppose the live support after fixed substitution is exactly one distinct row variable R and one distinct column variable C . Evaluating e at $(1, 1), (1, 0), (0, 1), (0, 0)$ and storing those four outputs in true-first order produces the unique token satisfying Equation (16).*

Theorem 4 (Soundness of admitted pair compilation). *Every derivable pair result with provenance $p \in \{S, T, H\}$ satisfies Equation (16). If no pair judgment is derivable, correctness is delegated to the unchanged ordinary IR evaluator.*

Proof. The cases S and T are the preceding theorem and lemma. For H , use structural induction with exact local tabulation as an additional sound base case; token complement and same-frame fusion preserve the invariant. Provenance changes the reported construction path, not the represented function. The fallback statement is an interface contract rather than a theorem about the pair compiler itself. $\square$

4.3 Admissibility boundary

The current pure structural rule does not claim semantic recognition of arbitrary equivalent compound operands. Repeated-variable forms such as $X \Theta X$, overlapping row/column layouts, opaque compound operands, and different variable pairs remain outside the pure rule. A retabulation rule may cover some two-variable cases exactly, but it is intentionally classified as a different mechanism.

This conservative boundary is useful. It allows the compiler to refuse a local rewrite rather than infer an unproved equivalence. It also gives the experiment a measurable fallback frequency instead of hiding unsupported cases in preprocessing.

Artifact	Meaning	Role
CM operator token	One binary truth function packed into four true-first bits (11,10,01,00).	Constant-size transform and fusion.
Pair surrogate	Token plus canonical row and column operand names/frame metadata.	Carries the legality invariant.
General CM/Boolean IR	Interned expression DAG for arbitrary supported formulas.	General symbolic fallback and compilation.
Flat program	Backend-dependent instruction sequence lowered from the IR.	Repeated packed/array evaluation.
Explicit dense output	Materialized truth relation over requested unfixed Boolean axes.	User-visible output; 2^n bits for n axes.

Table 1: Compiler artifacts and their semantic boundaries. Only the first is the constant-size binary operator token.

5 Implementation architecture and cost model

5.1 Five artifacts, not one

Table 1 separates implementation objects that are easily conflated when all are described informally as a “CM.”

A token-only endpoint cannot be compared as if it had already paid to construct an explicit 2^n -entry result. Conversely, a comparator that has already prepared a reusable DAG or program should not be charged for that preparation on every query when the CM arm is allowed resident reuse. Endpoint equality is part of benchmark correctness.

5.2 Executable case ordering

The implementation logic can be summarized as follows.

```

Pair(node e, frame Gamma, bool allowRetab):
  if e is a supported binary connective over signed literals:
    recognize row/column variables and finite negation chains
    normalize operand order and polarity into Gamma
    return structural Pair(R, C, token)

  if e = NOT a and Pair(a, Gamma, allowRetab) succeeds:
    complement token; preserve/upgrade provenance
    return Pair(...)

  if e = a Phi b and both recursive Pair calls succeed
  with identical canonical (R,C) metadata:
    fuse tokens entrywise under Phi
    return Pair(...)

  if allowRetab and live_support(e after fixed substitutions) = {R,C}:
    evaluate e on (1,1),(1,0),(0,1),(0,0)
    return retabulated Pair(R,C,token)

  return no-pair

TopLevel(e, strategy):
  pure_structural -> Pair(..., allowRetab=false) or ordinary IR
  hybrid          -> Pair(..., allowRetab=true) or ordinary IR
  retabulate      -> exact whole-root four-assignment token if admitted

```

Every recursive pair call receives a strict AST child, so structural recursion terminates on finite expressions. Retabulation performs four finite evaluations.

5.3 Layout conversion and ownership

The compact token uses true-first order while the historical dense array path uses false-first axes. The conversion is an explicit reindexing. If T denotes the true-first 2×2 view and D the false-first view indexed by array positions $a, b \in \{0, 1\}$, then

$$D_{ab} = T_{1-a, 1-b}. \quad (22)$$

Axes absent from the compact pair can then be broadcast into the requested output layout. The materialized dense result must be an owned output rather than a writable broadcast view. Expression nodes and pair surrogates are immutable under the compiler contract, and caller-owned layouts/fixed maps are read-only.

5.4 Cost model

Let N be AST occurrences, U unique DAG nodes, H AST height, S occurrences in a subtree sent to retabulation, D its evaluation depth, k the length of a signed literal’s negation chain, and n the number of requested unfixed output axes.

- Inspecting a signed literal costs $O(k)$ followed by an $O(1)$ four-bit transform.
- A four-bit token fusion is $O(1)$.
- One exact pair retabulation costs $O(4S)$ time plus the evaluator’s stack/workspace requirements.
- A fully successful structural pair tree is ordinarily $O(N)$ time and $O(H)$ stack space, although the current support-discovery strategy can rescan rejected subtrees and therefore has an $O(N^2)$ worst case.
- A sharing-aware evaluator may scale with U rather than N , which is why DAG-preserving packed/CSE comparators are mandatory.
- Materializing an arbitrary complete output over n unfixed Boolean axes costs $\Theta(2^n)$ time and output space for every explicit method.

The last point prevents a common category error: constant-time token fusion is a local kernel fact, not an end-to-end complexity theorem.

6 Closest prior art and matched comparators

The compiler should be compared with operations that possess equivalent local information, not with deliberately weaker baselines.

Boolean matrices and truth vectors. Edwards developed Boolean-matrix logic well before the present work [5]. Cheng, Zhao, and Xu use XOR as addition and AND as multiplication and state an entrywise outer-operation law on truth vectors of the same ordered variables [3]. Reshaping such a vector into 2×2 form does not make that pointwise identity new.

Matrix/vector logic. Stern and Mizraji provide established operator/matrix representations of logic [13, 8, 9]. Bricken’s technical note explicitly places logical matrices between two-component states in bra–matrix–ket form [1]. The companion foundations paper contains the appropriate broader comparison; the computational manuscript needs only enough of this history to prevent novelty from being attached to notation or representation alone.

NPN, LUT, and AIG practice. Small truth functions are routinely packed into machine words, transformed under input permutation/negation and output negation, classified up to NPN equivalence, and used in cut-based rewriting. DAG-aware AIG rewriting specifically exploits graph sharing rather than treating a formula as an unshared tree [7, 17, 14]. CM transposition, row/column reversal, complement, and four-bit storage are therefore best understood as a particular typed use of familiar small-function operations.

Direct packed evaluation. A strong simple comparator evaluates the AST once over four packed masks to obtain all four assignments in parallel. With identity memoization it also preserves explicit DAG sharing. It returns exactly the same four-bit function required by a pair-token endpoint and does not need CM-specific frame metadata. This is the closest baseline for deciding whether structural recognition pays for itself.

Generic local truth-function optimization. A generic optimizer given the same local support/function information can tabulate or combine a small cut without using CM notation. The S1/S2 evidence below shows why this comparator is scientifically necessary: it reaches the same reduced symbolic outputs as CM folding on the admitted all-arm cases.

ROBDDs and broader rewrite search. ROBDDs provide canonical function representations for a fixed variable order [2]. Equality saturation and synthesis-driven superoptimization retain or search broader classes of equivalent expressions [16, 12]. These systems solve a more general problem than the fixed two-operand rule; comparison should therefore charge recognition/search/extraction and compare delivered artifacts appropriate to the task.

7 Experimental questions and evidence hierarchy

The computational claim is meaningful only if the benchmark answers the same question for every arm. Four distinctions are especially important.

1. **Kernel versus compiler.** Four-bit fusion time is not compilation time.
2. **Prepared versus one-shot.** A resident token/program may amortize setup; one-shot use may not.
3. **Compact versus materialized output.** A token endpoint is not a complete 2^n truth relation.
4. **Task contract.** ANF production, complete truth-vector evaluation, repeated restriction, counting, and multi-root output are different tasks and cannot share one headline ratio.

Figure 2 shows the evidence levels used in this draft.

1. **Formal/finite correctness:** frame transforms, fusion invariant, retabulation, provenance, implementation tests.
2. **Exploratory mechanism:** S1/S2 pre-expansion work reductions and exact tie with matched generic local optimization.
3. **Completed different endpoint:** P14-PY0 ANF dispatcher, negative overall under its frozen primary rule.
4. **Separate project context:** later current-source complete-relation/restriction/multi-root campaigns; useful for methodology, not pair-compiler attribution.
5. **Open confirmatory claim:** protocol-v3 pair compiler versus direct/prepared packed, ROBDD, general IR, and applicable synthesis baseline.

Figure 2: Evidence hierarchy. A favorable result on a different endpoint is not promoted into pair-compiler evidence.

7.1 Confirmatory workload design

Protocol v3 separates exact-frame formulas, signed/swapped frames, local retabulation cases, hybrid mixed-child cases, multiple operand pairs, compound-operand cases outside the structural claim, and natural formulas/circuits. Balanced generated trees use AST occurrence counts 15, 63, 255, 1023, 4095; skewed trees stop at 255 occurrences and measured height at most 128. Cases cross the five supported outer connectives, reuse counts 1, 10, 100, 1000, and sharing conditions that distinguish identity-shared nodes, structurally equal but separately allocated subtrees, and no sharing. Every case records both N and U .

Requested outputs include compact tokens, scalar evaluation, packed batches, and dense outputs where the output budget permits them. Preparation and query costs are reported separately. Once two arms have produced the same four-bit token and call the same lookup function, the query cost is common; any advantage must arise earlier or through a measured reuse policy.

Corpus	Ratio	Geometric mean	98.75% simultaneous interval
Synthetic	E/F	0.9606	[0.9455, 0.9811]
Synthetic	E/D	1.0508	[0.9683, 1.1075]
Natural	E/F	0.9610	[0.9411, 1.0001]
Natural	E/D	1.0607	[1.0176, 1.0927]

Table 2: Frozen P14-PY0 recipe endpoint. Ratios below one favor the numerator.

The sole primary cell is intentionally narrow: signed/permutated pure-structural balanced formulas with $N = 1023$, $U = N$, no fixed variables, token output, reuse one, compared end to end with direct packed AST evaluation. The preregistered success rule requires zero correctness disagreements, a median competitor-to-CM ratio of at least 1.20 whose 95% interval lies entirely above one, and no more than a 10% median peak-memory increase. All other cells are secondary. This prevents a favorable post hoc workload from replacing a failed primary comparison.

8 Current evidence

8.1 Correctness and implementation assurance

The current implementation record reports 116 focused tests passing in an isolated source-copy run on 19 September 2026. The tested surface includes all 16 tokens in eight signed operand frames, the five supported primitive connectives, recursive fusion after alignment, pure-versus-hybrid provenance, whole-root retabulation, fixed substitutions in the direct packed comparator, true-first constants, 250 seeded random two-variable formulas, token-only API behavior, output-budget behavior, and input nonmutation. A separate exhaustive check enumerates 1,048,576 signed-alignment/fusion cases.

These counts are implementation assurance, not proofs of unbounded theorems and not evidence of speed. For submission, the test output, source hash, command, and environment should be bound into the same archival evidence package as the manuscript.

8.2 S1/S2: mechanism evidence and the generic tie

The canonical exploratory S1/S2 record contains 10,500 generated cases. Of these, 10,389 completed in all four arms: direct ANF, sharing-aware common-subexpression evaluation, a generic local truth-function optimizer, and CM folding. The all-arm population contains 3,750 S1 and 6,639 S2 cases. The remaining 111 are retained rather than silently filtered: 108 fail in all arms, two complete only in the generic and CM arms, and one only in the sharing arm. No completed correctness disagreement was recorded.

On every admitted all-arm case, CM and generic local optimization agree on seven principal symbolic-output metrics: final degree, final monomial count, multiplication count, output unique nodes, pair terms, peak monomials, and serialized bytes. The generic arm records more folds on 6,010 cases and CM more on none; fold count is not runtime. For corrected S2, median direct/CM multiplication counts at requested sharing fractions 0, 0.25, and 0.5 are respectively 35/35, 17/12, and 9/0.

The correct interpretation is therefore two-sided. Local small-function folding can remove work before polynomial expansion in some structures. The observed reduction is not uniquely CM-specific: an equally capable generic local optimizer reaches the same reduced outputs. This is mechanism evidence, not an end-to-end speed result.

8.3 P14-PY0: a completed negative dispatcher boundary

P14-PY0 measures a different endpoint: recipe-to-owned-ANF compilation. Direct lowering (D) lowers the ordinary source DAG; a minimal observer (F) records dispatch information; exact dispatch (E) decides whether to lower directly or construct folding metadata; always-fold (A) and a generic signature optimizer (G) are diagnostics. Preparation required by an arm is included in its production timer. This is not the direct packed token task of protocol v3.

The frozen campaign used 384 synthetic cases from 12 families and 107 natural cones from 14 previously uninspected EPFL design families, for 491 frozen cases. Three fresh sequential workers produced 51,102 measured rows including diagnostics/canonicalization; equality checks passed the 491 frozen cases and 512 targeted correctness cases. Exact dispatch selected folding on five synthetic cases and no natural cone.

The frozen primary rule required E/F geometric mean at most 0.99, E/D at most 0.95, and the relevant simultaneous upper bound below one on both corpora. It did not pass. At this endpoint exact dispatch was about 5.1% slower than direct lowering on the synthetic corpus and 6.1% slower on the natural corpus. The five selected synthetic cases themselves were highly profitable, with E/D ratios between 0.075 and 0.389, but those local wins did not offset aggregate policy overhead. The preregistered case-balanced geometric mean is retained even though an alternative ratio of summed synthetic medians was favorable. This is precisely the sort of negative boundary a selective compiler paper should report.

8.4 Later project-wide results: relevant context, different contracts

The broader public CM project continued benchmarking after the manuscript snapshot. Two later records are especially relevant to interpretation, but neither measures the protocol-v3 pair compiler.

First, a 25 August 2026 audit correction strengthened the generic comparator on a post-compilation kernel study. On its matched successor, sharing-aware CSE-flat and bare CM both reduced work relative to raw-AST execution. The formula-balanced bare-CM/CSE-flat ratio was 0.891 overall, narrowing to 0.961 at live support $k = 16$, while the public CM wrapper remained slower ($2.797\times$ CSE-flat overall and $1.343\times$ at $k = 16$) [10]. The project therefore distinguishes a favorable kernel boundary from an unfavorable one-shot wrapper boundary.

Second, a source-closed architecture comparison completed on 3–4 September 2026. For a complete explicit-relation task, a packed CM-IR arm was $1.053\times$ faster than dense CM but achieved only $0.930\times$ against the current identity-memoized direct BitSet arm; BitSet won all 78 runnable case clusters. In the corrected q1/q4/q16/q64 repeated-restriction ladder, Python R2 led at q1/q4 and CSE-flat bigint led at q16/q64 on the first host; at q64 CSE-flat was $1.100\times$ over R2, won all 54 cases, and retained a minimum above one. A separate host/compiler replication preserved the q1/q4 and q64 ordering, with CSE-flat at $1.090\times$ over R2 at q64. Native fused slots showed aggregate gains in some cohorts but unfavorable minima and remained guarded rather than universal defaults [11].

These later campaigns strengthen three methodological choices in the present paper: use direct packed evaluation as a first-class comparator, measure preparation/reuse explicitly, and keep task contracts separate. They do *not* fill the protocol-v3 pair-compiler result table.

8.5 The confirmatory gap

The broader token/packed protocol v3 remains unexecuted in the current manuscript record. Until that campaign is frozen and run, the paper cannot answer the strongest end-to-end question: whether typed structural recognition and frame normalization repay their costs against an equivalently capable direct packed/DAG baseline on the primary workload.

This gap is not hidden by substituting S1/S2, P14-PY0, or later project-wide architecture runs. Each of those records answers a different question.

9 Discussion

9.1 What the CM representation specifically contributes

The raw four truth values are not new. The computationally useful structure is the combination of a compact token with an explicit frame. That representation provides four practical properties.

First, *legality is local*: equality of canonical row/column metadata is a direct certificate for same-frame fusion. Second, signed normalization is fixed and cheap after recognition: transpose and row/column swaps act directly on four bits. Third, the fused result remains another four-bit token in the same frame, so the rewrite composes without expanding the operands. Fourth, provenance can say whether the token was derived structurally, retabulated, or assembled from both mechanisms.

A generic optimizer can implement the same semantic reduction with a different representation. The question is therefore not whether CM has exclusive access to the reduction, but whether the CM/frame organization lowers the engineering cost of recognizing and composing it in a larger compiler.

9.2 When folding can help

Folding is most plausible when the same two live operands recur through multiple signed/order variants, when expansion would otherwise repeat symbolic work, when the result will be reused, or when a local cut can be collapsed before an expensive downstream representation is built. The S1/S2 sharing strata provide a direct illustration: the largest observed reductions occur where repeated/local structure gives the small-function rewrite something to eliminate.

The same mechanism becomes less attractive when compatible pairs are rare, support discovery rescans rejected subtrees, a sharing-aware baseline evaluates unique DAG nodes once, a one-shot call cannot amortize preparation, or the requested output forces a much larger materialization. The later project-wide wrapper and BitSet results show that these overheads are not theoretical nuisances; they can dominate otherwise favorable kernel operations.

9.3 Retabulation is useful but conceptually different

Evaluating four assignments is exact for a two-variable function, but it should not be allowed to make a structural recognition rule look more powerful than it is. Retabulation consumes the subtree as a black box and reconstructs its local truth function. Structural folding instead proves that a token can be derived compositionally from recognized children and frame transforms. Both are valid compiler techniques; their costs and explanatory value differ.

This is why the hybrid path has its own provenance. It allows the practical compiler to continue after a local retabulation while preserving an auditable distinction between recognition and re-evaluation.

9.4 Sharing and output type change the break-even point

An AST occurrence count can dramatically overstate work when a comparator preserves identity sharing. Reporting both N and U is therefore mandatory. Equal-but-separately-allocated subtrees must also be distinguished from identity-shared nodes because hash-consing or structural CSE may discover the former while simple identity memoization does not.

Output type matters just as much. A four-bit token is a legitimate endpoint for a small-function compiler decision; it is not equivalent to returning a dense relation over many variables. Conversely, if a caller asks for repeated scalar or residual queries, preserving a prepared compact representation may be more important than materializing the full vector. Break-even reuse should therefore be derived from measured setup and per-query costs, not from an isolated token-fusion benchmark.

9.5 What a negative protocol-v3 result would mean

A negative confirmatory result would not invalidate the soundness theorem or the observation that local folding can reduce symbolic work. It would show that, under the tested implementation and workload distribution, recognition/planning/fallback costs erase those reductions relative to strong generic methods. That is a scientifically coherent outcome and would sharpen the contribution into a compiler-mechanism and evaluation-boundary paper.

Conversely, a positive result should remain scoped to the exact output, reuse, sharing, and workload region that passes the frozen rule. The later architecture campaigns demonstrate why a task map is more informative than a universal ratio.

10 Limitations

The current compiler intentionally recognizes a narrow class of two-operand local structure. It does not solve general Boolean equivalence, NPN canonicalization, LUT mapping, AIG rewriting, or broad superoptimization. Its support-discovery implementation can have a quadratic worst case on rejected structures. The implementation uses a host-language recursive traversal and therefore imposes an explicit height bound on skewed benchmark cells rather than relying on an increased recursion limit.

The S1/S2 corpus is synthetic/exploratory and contains only sixteen possible semantic functions on a fixed two-variable support; large two-variable trees measure syntactic redundancy and compiler behavior rather than growing semantic complexity. P14-PY0 measures ANF lowering rather than the packed-token endpoint. The later August/September project records use still other task contracts. Natural-workload prevalence of pure structural pair folding remains unresolved for this paper.

Finally, an explicit truth relation over n unfixed variables contains 2^n outputs. CM layout does not remove that information-theoretic output size. Compression, symbolic sharing, and partial-query benefits must be evaluated as separate representation or task properties.

11 Conclusion

The computational role of correspondence matrices is most defensible when stated narrowly. A four-bit binary operator token becomes useful to a compiler when its operand frame is typed. Signed and swapped occurrences can then be transported to a canonical frame, compatible tokens can be fused before operand evaluation or expansion, and unsupported structures can be retabulated or returned to an ordinary IR without compromising correctness.

The existing evidence draws an equally important boundary. Local small-function folding can reduce pre-expansion work, but a matched generic optimizer can reproduce the same symbolic reductions. A frozen dispatcher campaign shows that even very profitable local choices can lose overall once decision overhead is charged. Later project-wide benchmarks reinforce the need for direct packed/CSE baselines and task-matched endpoints, but they are not substitutes for the unexecuted pair-compiler protocol.

The next empirical question is therefore precise: on what workload, output, sharing, and reuse regimes does the typed frame-aware compiler repay its recognition and preparation costs? A completed protocol-v3 campaign can answer that question without requiring any claim that the underlying truth-table operations are uniquely CM-specific.

A Compact proof details

A.1 Signed transformations

Left multiplication by P exchanges the two rows of $[\Theta]$, so the row- x , column- y entry becomes the original row- $\neg x$, column- y entry, which is $(\neg x)\Theta y$. Right multiplication exchanges columns. Transposition exchanges the two assignments. Because P is a permutation matrix under XOR-AND multiplication, no multi-term cancellation occurs in these reindexings.

A.2 Hybrid induction

For pure structural derivations, primitive normalization is the base case and Equations (17)–(18) preserve Equation (16). For a hybrid derivation, exact local tabulation supplies an additional sound base case. Every parent above that base is again token complement or same-frame fusion. Thus hybrid provenance records how the token was obtained but does not alter its denotation.

A.3 Uniqueness of tabulated token

Fix R, C . Any binary Boolean function on those two variables is determined by its four values on $(1, 1), (1, 0), (0, 1), (0, 0)$. Storing those values in the declared true-first token order therefore produces the unique token that agrees with the source expression on every assignment.

B Current verification scope and submission checklist

The current manuscript/review artifacts report a focused 116-test run and an independent exhaustive signed-alignment/fusion enumeration of 1,048,576 cases. Before submission these should be re-executed from the same content-addressed source snapshot used for the final benchmark. The archival package should include the exact commands and stdout, source/environment hashes, natural-corpus provenance, raw S1/S2 and P14-PY0 records, protocol-v3 freeze and raw observations, ABC/AIG tool configuration for its admitted subset, analysis scripts, failures/timeouts, and a public archive identifier.

References

- [1] William Bricken. *Notes on Matrix Techniques for Logic*. Technical note internally dated March 1997; original public-release date and peer-reviewed status not established. <https://wbricken.com/pdfs/01bm/01math/03math-supporting/math-tangential/04matrix-tech.pdf>.
- [2] Randal E. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers*, C-35(8):677–691, 1986. doi:10.1109/TC.1986.1676819.
- [3] Daizhan Cheng, Yin Zhao, and Xiangru Xu. Matrix approach to Boolean calculus. In *Proceedings of the 50th IEEE Conference on Decision and Control and European Control Conference*, pages 6950–6955, 2011. doi:10.1109/CDC.2011.6160289.
- [4] Brian Droncheff. *Correspondence Matrices; Algorithms for Propositional Logic*. ResearchGate preprint, 2018. doi:10.13140/RG.2.2.28036.37764.
- [5] C. R. Edwards. The logic of Boolean matrices. *The Computer Journal*, 15(3):247–253, 1972. doi:10.1093/comjnl/15.3.247.
- [6] Winston Lee, Heinz Riener, Alan Mishchenko, Robert K. Brayton, and Giovanni De Micheli. A simulation-guided paradigm for logic synthesis and verification. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(8):2573–2586, 2022. doi:10.1109/TCAD.2021.3108704.
- [7] Alan Mishchenko, Satrajit Chatterjee, and Robert K. Brayton. DAG-aware AIG rewriting: A fresh look at combinational logic synthesis. In *Proceedings of the 43rd Design Automation Conference*, pages 532–536, 2006. doi:10.1145/1146909.1147048.
- [8] Eduardo Mizraji. Vector logics: The matrix–vector representation of logical calculus. *Fuzzy Sets and Systems*, 50(2):179–185, 1992. doi:10.1016/0165-0114(92)90216-Q.
- [9] Eduardo Mizraji. Vector logic: A natural algebraic representation of the fundamental logical gates. *Journal of Logic and Computation*, 18(1):97–121, 2008. doi:10.1093/logcom/exm057.
- [10] Correspondence Matrices Project. *CM benchmark audit correction — 2026-08-25*. Public technical record, 25 August 2026. https://github.com/Relative0/Correspondence_Matrices/blob/main/deliverables_n22_24/corrections_2026_08_25/CM_BENCHMARK_AUDIT_CORRECTION_REPORT_2026-08-25.md.
- [11] Correspondence Matrices Project. *Architecture-aware CM comparison refresh after C37*. Public technical record, 3 September 2026; updated 4 September 2026. https://github.com/Relative0/Correspondence_Matrices/blob/main/docs/research/CM_ARCHITECTURE_AWARE_COMPARISON_REFRESH_AFTER_C37_2026_09_03.md.
- [12] Raimondas Sasnauskas et al. Souper: A synthesizing superoptimizer. arXiv:1711.04422, 2017.
- [13] August Stern. *Matrix Logic: Theory and Applications*. North-Holland, 1988.
- [14] Eleonora Testa, Mathias Soeken, Luca Gaetano Amarù, and Giovanni De Micheli. Logic synthesis for established and emerging computing. *Proceedings of the IEEE*, 107(1):165–184, 2019. doi:10.1109/JPROC.2018.2869760.
- [15] Brian Theory. *Correspondence and Logical Matrices: A Boolean Operator Calculus: Formula-valued logical matrices, valuation, pairing, and coherent operator superposition*. Companion research draft, 22 September 2026.
- [16] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages* (POPL), 2021. arXiv:2004.03082.
- [17] Xuegong Zhou, Lingli Wang, and Alan Mishchenko. Fast adjustable NPN classification using generalized symmetries. *ACM Transactions on Reconfigurable Technology and Systems*, 12(2):7, 2019. doi:10.1145/3313917.